# Foundations of Machine Learning

## Course of Machine Learning

Giorgio Gambosi

Master's Degree in Computer Science
University of Rome "Tor Vergata"

A.Y. 2025–2026

# Outline

**Starting point**

Real-world data contains **hidden patterns** that reflect the process which generated it.

- Images, text, sensors, human behaviour: all exhibit *regular structure*.
- Hidden patterns emerge from the *generative process* underlying the data.
- The challenge of ML is to discover these patterns **automatically**, without being explicitly programmed with the governing rules.
- Discovery happens **by example**: we start from a collection of observations, each a snapshot of the phenomenon we wish to understand.

- From examples we build **models**: mathematical abstractions that capture relationships and patterns in the data.
- A model is a mathematical object (function, distribution, graph, ...) that summarises what we have learned.
- Models serve as tools for:
  - Prediction on new, unseen data.
  - Understanding the underlying phenomenon.
  - Decision-making under uncertainty.
- What matters is *not* how well the model fits the training data, but how well it **generalises**.

# The Three Learning Paradigms
## Overview

| Supervised | Unsupervised | Reinforcement |
|---|---|---|
| (Input, label) pairs.<br>Learn the mapping $\mathbf{x} \to t$. | Inputs only, no labels.<br>Explore intrinsic structure. | Agent + environment.<br>Learn through rewards. |

- Each paradigm poses distinct challenges and opportunities.
- This course focuses primarily on **supervised** and **unsupervised** learning.

**Intuitive example**

House dataset: area, rooms, location $\rightarrow$ selling price. We want to estimate the price of a *new* house.

- Data consists of pairs $(\mathbf{x}_i, t_i)$: input features and corresponding target value.
- The goal is to learn the relationship $\mathbf{x} \rightarrow t$ so we can generalise to new inputs.
- The final model is either a function $h(\mathbf{x})$ or a probability distribution $p(t|\mathbf{x})$.

### Regression

Target $t \in \mathbb{R}$ is continuous.

- House prices
- Future temperatures
- Stock values
- Drug dosages

### Classification

Target $t \in \mathcal{Y}$ is categorical.

- Spam / not spam ($|\mathcal{Y}| = 2$)
- Digits 0–9 ($|\mathcal{Y}| = 10$)
- Medical diagnosis
- Text sentiment

- **Binary** classification: $|\mathcal{Y}| = 2$.
- **Multi-class** classification: $|\mathcal{Y}| > 2$.

- **No labels available**: we explore the intrinsic structure of the data.
- **Clustering**: groups similar items without predefined categories; reveals natural segments.
- **Density estimation**: models $p_M(\mathbf{x})$, the distribution that generated the data.
- **Dimensionality reduction**: projects high-dimensional data ($d \gg 1$) onto compact subspaces while preserving essential information.

- **Generative modelling**: learn to *generate* new synthetic samples indistinguishable from real data.
  - Large Language Models (LLM)
  - Diffusion models (images, audio)
  - Generative Adversarial Networks (GAN)
- Requires having truly learned the deep structure of the data distribution.

**Further applications**

- **Anomaly detection**: points that deviate from the learned pattern.
- **Visualisation**: 2D/3D projections of complex data.
- **Feature engineering**: richer, more informative representations.

**Interaction cycle**

$$s_t \xrightarrow{\pi} a_t \longrightarrow r_t,\ s_{t+1} \longrightarrow \cdots$$

- An **agent** interacts sequentially with an environment.
- At each step it observes **state** $s$, selects **action** $a$, receives **reward** $r$ and new state $s'$.
- The **policy** $\pi : S \to A$ maps states to actions.
- Goal: find the optimal policy $\pi^*$ maximising the discounted cumulative reward:

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right], \qquad \gamma \in [0, 1].$$

# Reinforcement Learning
## Key tensions and applications

### Fundamental tradeoff: exploration vs. exploitation

- **Exploration**: try new actions to discover potentially better strategies.
- **Exploitation**: use known good actions to maximise immediate reward.
- Balancing both is essential for effective learning.

### Successful applications

- Games: AlphaGo defeated world Go champions.
- Robotics: learning motor skills and navigation.
- Autonomous systems: resource optimisation, scheduling.

**Definition**

A **feature vector** is $\mathbf{x} = (x_1, x_2, \ldots, x_d)$: a collection of $d$ numerical measurements describing one object. $d$ is called the **dimensionality**.

- Real-world entities (images, documents, patients, transactions) must be converted into numerical vectors.
- Features are the *raw material* from which models are built.
- The choice of features affects:
  - Prediction quality.
  - Computational efficiency of the learning algorithm.
  - Generalisation ability of the model.

# Data Representation
## Examples of representations

### Images

- Naïve: pixel intensity values (millions of features, noisy).
- Better: edge orientations, textures.
- Best: neural embeddings.

### Text

- Bag-of-words: word frequencies.
- TF-IDF: weighted relevance.
- Semantic embeddings (Word2Vec, BERT).

### Clinical data

Blood pressure, cholesterol, test results $\rightarrow$ vector of quantitative measurements.

- **Feature engineering**: defining new features via functional combinations of existing ones, selecting the most informative subset, or learning entirely new representations.
- Often *as important as* the choice of learning algorithm itself.

**Expert-guided**

A cardiologist knows which measurements matter for heart disease. Domain knowledge precedes data.

**Automated**

Unsupervised learning techniques autonomously discover the most informative representations (e.g. autoencoders, PCA).

# The Learning Process
## From training to generalisation

- We start from a **training set** $\mathcal{T} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)\}$.
- The learning algorithm builds a model that captures the patterns present in $\mathcal{T}$.
- The crucial point: we do *not* care how well the model fits the training data.

**The real objective: generalisation**

We want the model to perform well on **new, unseen data** — sampled from the same process that generated the training set.

# The Learning Process
## Key concepts

- **Capacity / Complexity**: how flexible the model is. Higher capacity can capture more intricate patterns, but risks overfitting.
- **Bias–variance tradeoff**: tension between underfitting (model too simple) and overfitting (model too adapted to training noise).
- **Regularisation**: techniques that penalise model complexity, encouraging simpler solutions that generalise better.
- **Generalisation**: the ability to transfer learned knowledge to new situations.

# The Learning Process
## The role of models

- Models are the bridge between data and prediction: they embody what we have learned from the training set.
- In **supervised** learning: the model captures the input–output relationship, enabling predictions on new inputs.
- In **unsupervised** learning: the model captures the structure of the data distribution.
- Possible forms of a model:
  - Linear function.
  - Deep neural network.
  - Probabilistic distribution.
  - Tree ensemble (e.g. Random Forest).
- What unites them all: building from data something that *transfers knowledge* to new situations.

# Formal Definition
## Domain set and Label set

### Domain set $\mathcal{X}$

Set of objects we wish to label.

- Each $\mathbf{x} \in \mathcal{X}$ is a vector of $d$ features.
- $\mathbf{x} = (x_1, \ldots, x_d)$; $d =$ dimensionality.
- More complex structures (sequences, graphs) are also possible.

### Label set $\mathcal{Y}$

Set of possible output values.

- $\mathcal{Y} \subseteq \mathbb{R}$ (continuous) $\Rightarrow$ **regression**.
- $\mathcal{Y}$ finite $\Rightarrow$ **classification**.
- $|\mathcal{Y}| = 2$: binary.
- $|\mathcal{Y}| > 2$: multi-class.

# Formal Definition

## Training set and data structures

- The learning algorithm $\mathcal{A}$ receives a training set: $\mathcal{T} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_n, t_n)\}$
- Organised as:
  - **Feature matrix** $\mathbf{X} \in \mathbb{R}^{n \times d}$: objects as rows.
  - **Target vector** $\mathbf{t} \in \mathcal{Y}^n$: corresponding labels.
- Output of the algorithm: a **prediction rule** $h : \mathcal{X} \to \mathcal{Y}$ (also called *classifier* or *regressor*).

**Goal**

$h$ must correctly predict label $y$ for any new object $\mathbf{x}$ it encounters.

# Formal Definition

## Two output strategies

### Direct value prediction

$y = h(\mathbf{x})$

- Single predicted label returned directly.
- Simple and computationally efficient.
- Does not quantify uncertainty.

### Probability distribution

$p(y|\mathbf{x})$ for each $y \in \mathcal{Y}$

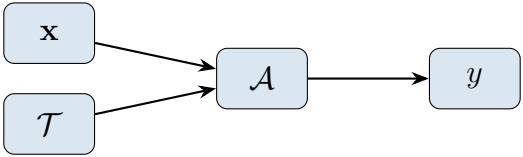
- Estimates probability of each label.
- Richer: captures uncertainty.
- Final prediction via decision rule, e.g. $\hat{y} = \arg\max_y p(y|\mathbf{x})$.

# Three Approaches
## Overview

| Approach | Core idea | Example |
|---|---|---|
| Direct prediction | Entire training set used at prediction time | $k$-NN |
| Model-based | Learn $h_{\mathcal{T}} \in \mathcal{H}$ during a training phase | Linear regression |
| Ensemble | Combine multiple weighted predictors | Random Forest |

- Three distinct philosophies about *when* and *how* to process the training data.

# Approach 1: Direct Prediction
## Data-flow diagram




- Both the new point $\mathbf{x}$ and the entire training set $\mathcal{T}$ are inputs to the algorithm $\mathcal{A}$ at prediction time.
- No separate learning phase; no compact model is built.
- All knowledge stays in the raw training examples.

# Approach 1: Direct Prediction

## $k$-Nearest Neighbours

### kNN algorithm

To predict the class of a new point $\mathbf{x}$:

1. Find the $k$ training points closest to $\mathbf{x}$ under a distance metric (e.g. Euclidean).
2. Predict the majority class among those $k$ neighbours.

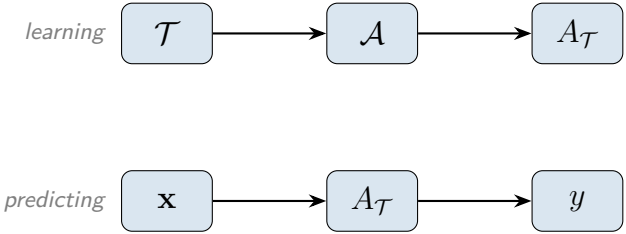
### Advantages

- Flexible, locally adaptive.
- No training phase.
- Simple to implement.

### Disadvantages

- Costly at prediction time: $\mathcal{O}(n)$.
- Must store entire $\mathcal{T}$.
- Suffers from the *curse of dimensionality*.

# Approach 2: Model-Based Learning
## Data-flow diagram




- **Learning phase**: algorithm $\mathcal{A}$ maps $\mathcal{T} \to A_{\mathcal{T}}$ (a fixed learned function).
- **Prediction phase**: $A_{\mathcal{T}}$ maps new $\mathbf{x} \to y$ efficiently, with no access to $\mathcal{T}$.

# Approach 2: Model-Based Learning
## Hypothesis class

- Define a **hypothesis class** $\mathcal{H}$: a predefined family of functions $h : \mathcal{X} \to \mathcal{Y}$.
- The learning algorithm $\mathcal{A}$ selects the function $h_{\mathcal{T}} \in \mathcal{H}$ that best explains the training data.
- Once $h_{\mathcal{T}}$ is learned, prediction is immediate: $y = h_{\mathcal{T}}(\mathbf{x})$.

**Parametric classes**

Often $\mathcal{H} = \mathcal{H}_{\boldsymbol{\theta}}$: functions with the same structure but differing parameters.
Learning $\equiv$ finding the optimal $\boldsymbol{\theta}^* = (\theta_1, \ldots, \theta_m)$.

# Approach 2: Model-Based Learning
## Example: Linear Regression

### Linear regression

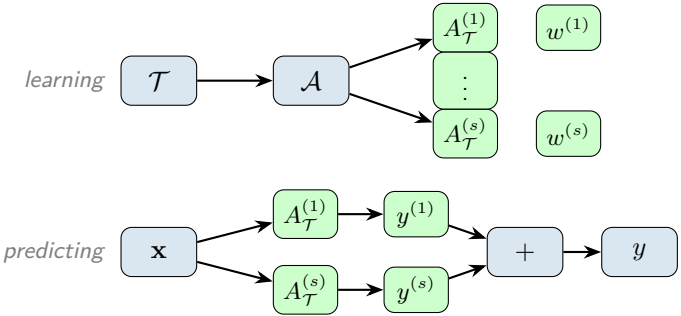
Assume the target is a linear combination of features:

$$y = \sum_{i=1}^{d} w_i x_i + w_0 = \mathbf{w}^\top \bar{\mathbf{x}}$$

where $\mathbf{w} = (w_0, w_1, \ldots, w_d)^\top$ and $\bar{\mathbf{x}} = (1, x_1, \ldots, x_d)^\top$.

- $\mathcal{H}$ = all $(d+1)$-dimensional linear functions.
- The algorithm finds weights $\mathbf{w}$ minimising prediction error on the training set.
- **Advantages**: efficient at prediction time ($\mathcal{O}(d)$), interpretable, compact.

# Approach 3: Ensemble Learning
## Data-flow diagram



# Approach 3: Ensemble Learning
## General principle

- Rather than searching for a single best predictor, build $s$ **predictors** and combine their outputs.
- Predictors may differ in structure, parameters, or the subset of training data used.
- Each is assigned a weight $w^{(i)} \geq 0$ reflecting its estimated reliability, with $\sum_{i=1}^{s} w^{(i)} = 1$.

**The three-stage process**

1. **Learning**: derive $h_{\mathcal{T}}^{(1)}, \ldots, h_{\mathcal{T}}^{(s)}$ from $\mathcal{T}$.
2. **Weighting**: assign weights $w^{(i)}$.
3. **Combination**: aggregate predictions at test time.

# Approach 3: Ensemble Learning
## Combining predictions

**Regression: weighted average**

$$h(\mathbf{x}) = \sum_{i=1}^{s} w^{(i)}\, h^{(i)}(\mathbf{x})$$

Interpretable as a weighted mean.

**Classification: weighted vote**

$$h(\mathbf{x}) = \arg\max_{y} \sum_{i=1}^{s} w^{(i)}\, \mathbf{1}\left[h^{(i)}(\mathbf{x}) = y\right]$$

The class with most aggregated weight wins.

- **Fully Bayesian** variant: continuous average over all parameter values, weighted by the posterior distribution.
- **Advantages**: robust to noise, often more accurate than any individual model.

# Three Approaches: Comparison

## When to use each

| | **Direct** | **Model-Based** | **Ensemble** |
|---|---|---|---|
| Training cost | None | High | Very high |
| Prediction cost | $O(n)$ | $O(d)$ (fast) | Moderate |
| Interpretability | Low | High | Medium/Low |
| Accuracy | Locally good | Depends on $\mathcal{H}$ | Generally high |

# Probabilistic Framework
## Why a probabilistic view is needed

- Real data is *noisy*: the same $\mathbf{x}$ may carry different labels across observations.
- Sources: measurement noise, inherent ambiguity, labelling subjectivity.
- A probabilistic framework explicitly captures this uncertainty.

**The two core assumptions**

- Objects are sampled i.i.d. from an unknown distribution $p_M(\mathbf{x})$ over domain $\mathcal{X}$.
- Labels arise from a conditional distribution $p_C(t|\mathbf{x})$: given $\mathbf{x}$, $t$ is *not* deterministic.

# Probabilistic Framework

## Joint distribution and deterministic case

- The two assumptions induce a **joint distribution**:

$$p(\mathbf{x}, t) = p_M(\mathbf{x}) \cdot p_C(t|\mathbf{x})$$

- Captures how often we observe object–label pairs.

**Simplified case: deterministic ground truth**

We often assume an unknown function $f : \mathcal{X} \to \mathcal{Y}$ such that $t = f(\mathbf{x})$.

- Equivalent to $p_C(t|\mathbf{x}) = 1$ if $t = f(\mathbf{x})$, $0$ otherwise.
- Conceptually cleaner, but less realistic.
- The probabilistic framework is the general case.

# Loss Function

## Measuring the error

**Definition**

The **loss function** $L : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ quantifies the penalty of predicting $y$ when the true label is $t$.

**0-1 loss (classification)**

$$L(y, t) = \mathbf{1}[y \neq t]$$

Penalty 1 for any error, 0 if correct. Intuitive but not differentiable.

**Squared loss (regression)**

$$L(y, t) = (y - t)^2$$

Penalises large errors more heavily. Differentiable; suitable for optimisation.

- The choice of loss influences both the optimisation algorithm and the properties of the learned model.

# Prediction Risk
## For a single object

- The **prediction risk** $\mathcal{R}(y, \mathbf{x})$ measures the expected loss for a single object $\mathbf{x}$.

**Deterministic case ($t = f(\mathbf{x})$)**

$$\mathcal{R}_f(y, \mathbf{x}) \;=\; L\big(y,\; f(\mathbf{x})\big)$$

**Probabilistic case**

$$\mathcal{R}_{p_C}(y, \mathbf{x}) = \mathbb{E}_{t \sim p_C(\cdot|\mathbf{x})}\big[L(y, t)\big] = \sum_{t \in \mathcal{Y}} L(y, t) \cdot p_C(t|\mathbf{x})$$

(sum for discrete $\mathcal{Y}$; integral for continuous $\mathcal{Y}$)

- Lower risk $\Leftrightarrow$ better alignment with the true label distribution.

# Expected Risk (True Risk)

## The ideal performance measure

- To evaluate $h$ *globally*, aggregate the prediction risk over all objects, weighted by $p_M$.

**Definition (deterministic case)**

$$\mathcal{R}_{p_M,f}(h) = \mathbb{E}_{\mathbf{x}\sim p_M}\big[L(h(\mathbf{x}), f(\mathbf{x}))\big] = \int_{\mathcal{X}} L\big(h(\mathbf{x}), f(\mathbf{x})\big)\, p_M(\mathbf{x})\, d\mathbf{x}$$

**With 0-1 loss: intuitive interpretation**

$$\mathcal{R}_{p_M,f}(h) = \Pr_{\mathbf{x}\sim p_M}\big[h(\mathbf{x}) \neq f(\mathbf{x})\big]$$

The expected risk equals the **probability of error** on a randomly drawn object.

# Empirical Risk
## The computable surrogate

### The fundamental problem

$p_M$ and $f$ (or $p_C$) are **unknown and unknowable**. We cannot directly minimise the expected risk.

### Solution: empirical risk

Estimate the true risk using only the available training data:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L\big(h(\mathbf{x}), t\big)$$

- With 0-1 loss: fraction of misclassified examples (**error rate**).
- The empirical risk *estimates* the true risk and converges to it as $n \to \infty$.

# Empirical Risk Minimisation (ERM)

## The learning paradigm

### ERM formalisation

Given a candidate set $\mathcal{H}$, find:

$$h_{\mathcal{T}} = \arg\min_{h \in \mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h)$$

The predictor in $\mathcal{H}$ that minimises the empirical risk.

- $\mathcal{H}$ is also called the **inductive bias**: it encodes assumptions about which functions are plausible.
- Restricting to $\mathcal{H}$ is an implicit statement about the structure of the problem.

### Warning

Minimising empirical risk does *not* guarantee good generalisation. Empirical risk is only a *proxy* for the true risk.

# Inductive Bias
## Choosing $\mathcal{H}$

- The choice of $\mathcal{H}$ is one of the most consequential decisions in ML.
- $\mathcal{H}$ embodies the **inductive bias**: assumptions about which type of predictor is likely to work well.
- Choosing $\mathcal{H}$ means saying: *"Somewhere in this class there exists a function that will generalise well to unseen data."*

**$\mathcal{H}$ too small**

Risk of excluding the true function. Even with infinite data, error stays high.

**$\mathcal{H}$ too large**

The algorithm finds functions that fit training data perfectly but fail on new data.

# Overfitting
## When training success masks failure

**Example: the memoriser**

With $\mathcal{H} =$ all functions from $\mathcal{X}$ to $\{0, 1\}$, consider the predictor:

$$h_{\mathcal{T}}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{x}_i \in \mathbf{X} \text{ and } t_i = 1 \\ 0 & \text{otherwise} \end{cases}$$

- On training set: empirical risk $= 0$. Looks perfect.
- On new data: the probability that $\mathbf{x} \in \mathcal{T}$ is negligible $\Rightarrow$ predictor always outputs 0.
- With balanced classes: expected risk $\approx \frac{1}{2}$ (random guessing).

**Definition of overfitting**

The gap between training and test performance. The model memorised the data without capturing any generalisable pattern.

- Occurs when $\mathcal{H}$ is so large that the algorithm finds functions with low empirical risk but high expected risk.
- The model learns the *noise* in the training set, not the *signal*.
- Symptom: **low training error, high test error**.

# Underfitting

## When $\mathcal{H}$ is too restrictive

- Opposite problem: $\mathcal{H}$ too small to capture the true underlying function.
- Example: a linear model on highly non-linear data — regardless of data quantity, error stays high.

**Distinguishing characteristic**

With underfitting there is **no gap** between training and test error: both remain unacceptably high.

- The model is too rigid to memorise anything.
- It fails to fit even the training data.

# Bias and Variance
## Definitions

- Consider training sets $\mathcal{T}_1, \mathcal{T}_2, \mathcal{T}_3, \ldots$ repeatedly drawn from the same distribution $p_M$.
- Algorithm $\mathcal{A}$ produces a different predictor $h_{\mathcal{T}_1}, h_{\mathcal{T}_2}, h_{\mathcal{T}_3}, \ldots$ for each.

**Bias**

How far the *average* predictor (over many training sets) deviates from the true $f$.

**Systematic error**, independent of the particular training set received.

**Variance**

How much the predictor *fluctuates* with the particular training set received.

**Sensitivity to noise** in the training data.

# Bias and Variance
## Causes and relation to $\mathcal{H}$

- **High bias** $\leftarrow \mathcal{H}$ too restrictive: a linear model on non-linear data will always have high bias, regardless of training-data size.
- **High variance** $\leftarrow \mathcal{H}$ too flexible: small changes in the training set produce dramatically different learned functions. Signature of overfitting.

**The fundamental tension**

- Small $\mathcal{H}$ (e.g. linear functions): **low variance**, but **high bias**.
- Large $\mathcal{H}$ (e.g. all possible functions): bias near zero, but **very high variance**.

# The Bias–Variance Tradeoff

## Cannot minimise both simultaneously

**The tradeoff is intrinsic**

Bias and variance cannot be minimised simultaneously. There exists an optimal size for $\mathcal{H}$ that balances the two.

- **Too small**: bias dominates.
- **Too large**: variance dominates.
- The optimal point depends on:
  - True complexity of the underlying function.
  - Number $n$ of training examples available.
  - Noise level in the observations.
- *The art of ML lies in choosing an inductive bias that matches the actual complexity of the task.*

# The Bias–Variance Tradeoff

## Behaviour as $n$ grows

- With **small** training sets: a flexible model can still perform well if it captures the true pattern before fitting the noise.
- As $n$ grows: flexible models start to overfit (more data with different noise patterns to memorise).
- Restrictive models never improve enough, regardless of how much data is added.

### Learning curves (schematic)

| Model | Training error | Test error |
|---|---|---|
| Too simple | High | High (same level) |
| Optimal | Low | Low |
| Too complex | Very low | High |

# Computational Considerations
## An additional dimension of choice

- Even with a statistically optimal $\mathcal{H}$, minimising $\overline{\mathcal{R}}_{\mathcal{T}}$ over $\mathcal{H}$ may be **computationally intractable**.
- Some hypothesis classes, though statistically sound, lead to NP-hard optimisation problems.
- In such cases we deliberately choose a smaller class $\mathcal{H}'$ to make the problem solvable in reasonable time.

**Inductive bias = statistics + computation**

The choice of $\mathcal{H}$ reflects a compromise between:

- **Statistical assumptions** about which function is plausible.
- **Computational constraints** on what can be computed in reasonable time.

# Summary
## Core concepts

### Main concepts

- Paradigms: supervised, unsupervised, RL.
- Feature vectors and feature engineering.
- Training set and generalisation.
- Three approaches to deriving a predictor.

### Formal framework

- Loss function $L(y, t)$.
- Expected risk $\mathcal{R}(h)$.
- Empirical risk $\overline{\mathcal{R}}_{\mathcal{T}}(h)$.
- ERM: $h_{\mathcal{T}} = \arg\min_{h \in \mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}$.

### Central message

The goal of ML is *not* to fit the training data, but to **generalise** to new data. The choice of inductive bias $\mathcal{H}$ is the critical decision for balancing bias, variance and computational cost.

# Summary
## Key formulas at a glance

| Concept | Formula |
|---|---|
| 0-1 loss | $L(y,t) = \mathbf{1}[y \neq t]$ |
| Squared loss | $L(y,t) = (y-t)^2$ |
| Prediction risk (det.) | $\mathcal{R}_f(y,\mathbf{x}) = L(y, f(\mathbf{x}))$ |
| Expected risk | $\mathcal{R}_{p_M,f}(h) = \mathbb{E}_{\mathbf{x}\sim p_M}[L(h(\mathbf{x}), f(\mathbf{x}))]$ |
| Empirical risk | $\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{\lvert\mathcal{T}\rvert}\sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}),t)$ |
| ERM | $h_{\mathcal{T}} = \arg\min_{h\in\mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h)$ |
| RL objective | $\pi^* = \arg\max_\pi \mathbb{E}_\pi[\sum_{t\geq 0}\gamma^t r_t]$ |

# Introduction to Statistical Learning Theory

## Course of Machine Learning

Giorgio Gambosi

Master's Degree in Computer Science
University of Rome "Tor Vergata"

A.Y. 2025–2026

# Outline

**What Statistical Learning Theory asks**

Given a hypothesis class $\mathcal{H}$ and a training set of size $n$:

- What can we **guarantee** about the quality of the learned predictor?
- Under what conditions is learning **actually possible**?

- Not about specific algorithms or datasets.
- About the *general structure* of the learning problem.
- Provides **precise mathematical answers** — universal principles.
- Reveals *whether* and *how quickly* we can learn.

# Introduction
## Learning algorithms and hypothesis classes

- A learning algorithm $\mathcal{A}$ takes training data and produces a predictor.
- $\mathcal{A}$ must operate within a structured framework: the **hypothesis class** $\mathcal{H}$.
- $\mathcal{H}$ = set of candidate functions the algorithm is allowed to choose from.

**Inductive bias**

The hypothesis class embodies the algorithm's **inductive bias**: prior beliefs about what kind of solution is likely to work.

Every learning algorithm must encode this bias — explicitly or implicitly.

- Notation: $h_{\mathcal{T}}^{\mathcal{A}}$ = predictor returned by $\mathcal{A}$ on training set $\mathcal{T}$.
- The learned predictor depends on *both* the algorithm and the training data.

### ERM — the canonical learning algorithm

Given $\mathcal{T}$ and $\mathcal{H}$, return the predictor with lowest training error:

$$h_{\mathcal{T}} = \arg\min_{h \in \mathcal{H}} \overline{\mathcal{R}}_{\mathcal{T}}(h)$$

where $\overline{\mathcal{R}}_{\mathcal{T}}(h)$ is the empirical risk — average loss on the training set.

- Intuition: choose the predictor that works best on available data.
- Danger: the training data may be *misleading*.
- A predictor can fit perfectly yet generalise poorly (**overfitting**).

### Central question of SLT

When does minimising training error lead to low **true** error?
The answer depends critically on the hypothesis class.

**What determines whether learning is possible?**

1. The **size** of the hypothesis class (how many functions it contains).
2. The **amount** of training data available.
3. The required **accuracy and confidence** (our tolerance for error).

- Some classes are *learnable*: with enough data we find good predictors.
- Some classes are *too complex*: we always find functions fitting training data while failing on new data.
- Understanding these relationships is the **essence of SLT**.

# Finite Classes under Realizability
## Setup

- We consider **binary classification** with 0-1 loss throughout this section.
- We assume the hypothesis class $\mathcal{H}$ is **finite**: $|\mathcal{H}| < \infty$.
- Simplest guarantee: if $|\mathcal{H}|$ is bounded and $n$ is large enough, ERM does not overfit.

**Realizability assumption**

There exists $h^* \in \mathcal{H}$ such that:

$$\mathcal{R}_{p_M,f}(h^*) = \mathbb{E}_{p_M,f}[|\mathbf{x} \in \mathcal{X} : h^*(\mathbf{x}) \neq f(\mathbf{x})|] = 0$$

$h^*$ makes **no errors** on the entire population $\mathcal{X}$.

- Strong assumption — idealised setting.
- Isolates the challenge of learning from *finite samples*.

# Finite Classes under Realizability
## ERM under realizability

- Under realizability, $h^*$ correctly classifies *every* element of $\mathcal{X}$.
- In particular, for any training set $\mathcal{T} = (\mathbf{X}, \mathbf{t})$ with $t_i = f(\mathbf{x}_i)$:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h^*) = \frac{|\{(\mathbf{x}, t) \in \mathcal{T} : h^*(\mathbf{x}) \neq t\}|}{|\mathcal{T}|} = 0$$

**ERM result under realizability**

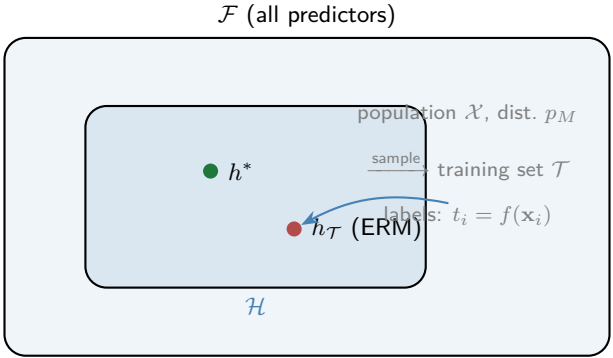
Since $h^*$ has zero training error, ERM will return *some* predictor $h_{\mathcal{T}}$ also with zero training error:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) = 0$$

- **Caution**: $h_{\mathcal{T}}$ may differ from $h^*$.
- Zero training error *does not* guarantee zero true error.
- Other functions in $\mathcal{H}$ may also achieve zero training error — some may be bad.

# Finite Classes under Realizability

## The learning scenario (figure)




*ERM selects $h_{\mathcal{T}} \in \mathcal{H}$ minimising empirical risk. Under realizability $\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) = 0$, but true risk $\mathcal{R}_{p_M,f}(h_{\mathcal{T}})$ may be positive if $h_{\mathcal{T}} \neq h^*$.*

# Finite Classes under Realizability

## Bad predictors and bad datasets

### Bad predictor

A predictor $h \in \mathcal{H}$ is **bad** if its true risk exceeds the tolerance $\varepsilon > 0$:

$$\mathcal{R}_{p_M,f}(h) > \varepsilon$$

Let $\mathcal{H}_B$ denote the set of all bad predictors.

### Bad (misleading) training set

A training set $\mathcal{T}$ is **bad** if at least one bad predictor achieves zero training error on it:

$$\exists\, h \in \mathcal{H}_B : \ \overline{\mathcal{R}}_{\mathcal{T}}(h) = 0$$

Bad training sets create the *illusion* that a poor predictor is good.

- Goal: bound the probability of drawing a bad training set.

**Step 1 — single bad predictor.** A bad $h$ misclassifies $\geq \varepsilon$ fraction of the population.

$$\Pr[h \text{ gets one random point right}] \leq 1 - \varepsilon$$

For $n$ i.i.d. points:

$$\Pr[\overline{\mathcal{R}}_{\mathcal{T}}(h) = 0] \leq (1-\varepsilon)^n \leq e^{-\varepsilon n}$$

using $1 - x \leq e^{-x}$.

**Step 2 — any bad predictor (union bound).**

$$\mathbb{P}_{\mathcal{T}\sim p^n}\left[\exists\, h \in \mathcal{H}_B :\ \overline{\mathcal{R}}_{\mathcal{T}}(h) = 0\right] \leq \sum_{h\in\mathcal{H}_B} e^{-\varepsilon n} \leq |\mathcal{H}| \cdot e^{-\varepsilon n}$$

**Step 3 — set the bound $\leq \delta$.**

Require $|\mathcal{H}| \cdot e^{-\varepsilon n} \leq \delta$, i.e. $e^{-\varepsilon n} \leq \dfrac{\delta}{|\mathcal{H}|}$.

Taking logarithms of $e^{-\varepsilon n} \leq \dfrac{\delta}{|\mathcal{H}|}$:

$$-\varepsilon n \leq \ln \frac{\delta}{|\mathcal{H}|} \quad \Longrightarrow \quad n \geq \frac{1}{\varepsilon}(\ln |\mathcal{H}| - \ln \delta) = \frac{1}{\varepsilon} \ln \frac{|\mathcal{H}|}{\delta}$$

**Sample complexity — finite class, realizability**

$$n \geq \left\lceil \frac{1}{\varepsilon} \ln \frac{|\mathcal{H}|}{\delta} \right\rceil$$

With this many examples, with probability $\geq 1 - \delta$, ERM returns a predictor with true risk $\leq \varepsilon$.

- **Accuracy** $\varepsilon$: inverse linear — halving $\varepsilon$ doubles $n$.
- **Confidence** $\delta$: logarithmic — high confidence is cheap.
- **Class size** $|\mathcal{H}|$: logarithmic — even huge finite classes are learnable.

# Finite Classes under Realizability

## Interpreting the components

### Accuracy $\varepsilon$

Denominator $\Rightarrow$ inverse relation.

$\varepsilon/2$ needs $2\times$ data.

**Linear cost** for accuracy.

### Confidence $\delta$

Inside a log $\Rightarrow$ logarithmic.

$10\times$ higher confidence: add $\frac{\ln 10}{\varepsilon}$ examples.

**Cheap** to increase confidence.

### Class size $|\mathcal{H}|$

$\ln|\mathcal{H}| \Rightarrow$ logarithmic.

$|\mathcal{H}|$ from $100$ to $10^6$: add $\frac{\ln 10^4}{\varepsilon}$ examples.

**Huge** classes still learnable.

### Numerical example

$|\mathcal{H}| = 10{,}000,\ \ \varepsilon = 0.1,\ \ \delta = 0.05$: $\quad n \geq \frac{1}{0.1}\ln\frac{10000}{0.05} \approx 122$ examples.

Improve to $\varepsilon = 0.02$: $\quad n \approx 610$ — five times more data for five times better accuracy.

# Finite Classes under Realizability

## Bad vs. very bad datasets

**Bad dataset**

At least one bad predictor $h \in \mathcal{H}_B$ achieves $\overline{\mathcal{R}}_{\mathcal{T}}(h) = 0$.

ERM *might still* return a good predictor if another zero-error predictor is also present and chosen.

**Very bad dataset**

ERM *specifically* returns a bad predictor.

Worst case for the analysis — the union bound is deliberately pessimistic to cover this scenario.

- The bound controls probability of bad datasets.
- In practice, ERM often picks a good predictor among zero-error ones.
- The union bound guarantees correctness in the **worst case**.

**Why finiteness is crucial**

The union bound sums probabilities over $|\mathcal{H}|$ predictors.
For **infinite** $\mathcal{H}$: the bound becomes $\infty$ — useless.

**Finite $\mathcal{H}$**

**Always learnable.**

Sample complexity grows only as $\ln |\mathcal{H}|$.

Even classes with trillions of functions are learnable.

**Infinite $\mathcal{H}$**

**Requires different analysis.**

Cannot count functions.

Need complexity measures like **VC dimension** or Rademacher complexity.

- This is why SLT focuses on *characterising* hypothesis class complexity.

# PAC Learning
## Probably Approximately Correct

### Motivation

**PAC learning** formalises what it means for a learning problem to be solvable. It answers: which hypothesis classes are learnable, and how much data is needed?

The framework captures two sources of imperfection:

- **"Approximately"**: we don't demand perfect performance — only error $\leq \varepsilon$.
- **"Probably"**: we don't need success for every training set — only with probability $\geq 1 - \delta$.

- $\varepsilon \in (0, 1)$: **accuracy** parameter.
- $\delta \in (0, 1)$: **confidence** parameter.
- $m_{\mathcal{H}}(\varepsilon, \delta)$: **sample complexity** — data required.

**Definition 1 (PAC Learnability)**

*A hypothesis class $\mathcal{H}$ is* ***PAC learnable*** *if there exist a sample complexity function $m_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ and a learning algorithm $\mathcal{A}$ such that:*
*For every $\varepsilon, \delta \in (0,1)$, every distribution $p_M$ over $\mathcal{X}$, and every labelling $f : \mathcal{X} \to \{0,1\}$ satisfying the realizability assumption, if $\mathcal{A}$ is applied to $n \geq m_{\mathcal{H}}(\varepsilon, \delta)$ i.i.d. examples, it returns $h_{\mathcal{T}}$ satisfying*

$$\mathcal{R}_{p_M, f}(h_{\mathcal{T}}) \leq \varepsilon$$

*with probability at least $1 - \delta$ over the random training set.*

- For finite classes, ERM is a PAC learner with complexity $m_{\mathcal{H}}(\varepsilon, \delta) \leq \left\lceil \frac{1}{\varepsilon} \ln \frac{|\mathcal{H}|}{\delta} \right\rceil$.

### Optimal predictor under stochastic labels

When labels are generated by $p_C(t|\mathbf{x})$, the optimal predictor is:

$$h^*(\mathbf{x}) = \arg\min_{y \in \{0,1\}} \mathbb{E}_{t \sim p_C(\cdot|\mathbf{x})}[L(y,t)]$$

For binary classification with 0-1 loss, this simplifies to:

$$h_{\text{Bayes}}(\mathbf{x}) = \begin{cases} 1 & \text{if } p_C(t=1|\mathbf{x}) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

- $h_{\text{Bayes}}$ is the **Bayes predictor** — best possible classifier.
- Its risk $\mathcal{R}^* = \mathcal{R}_p(h_{\text{Bayes}})$ is the **irreducible error** — no algorithm can do better.
- **Unattainable** in practice: requires knowing $p_C(t|\mathbf{x})$.

### Motivation

Realizability rarely holds. We need a framework that does *not* assume our class contains a perfect predictor.

### Definition 2 (Agnostic PAC Learnability)

*$\mathcal{H}$ is **agnostic PAC learnable** if there exist $m_{\mathcal{H}}$ and $\mathcal{A}$ such that for every $\varepsilon, \delta \in (0,1)$ and every distribution $p$ over $\mathcal{X} \times \mathcal{Y}$, applying $\mathcal{A}$ to $n \geq m_{\mathcal{H}}(\varepsilon, \delta)$ i.i.d. examples returns $h_{\mathcal{T}}$ satisfying*

$$\mathcal{R}_p(h_{\mathcal{T}}) \leq \min_{h \in \mathcal{H}} \mathcal{R}_p(h) + \varepsilon$$

*with probability at least $1 - \delta$.*

- We compare to the **best achievable in** $\mathcal{H}$, not to zero error.
- Generalises PAC: if realizability holds, the definition reduces to ordinary PAC.

### Two sources of error

$$\mathcal{R}_p(h_{\mathcal{T}}) - \mathcal{R}^* = \underbrace{\left(\mathcal{R}_p(h_{\mathcal{T}}) - \mathcal{R}_p(h^*)\right)}_{\text{estimation error } \varepsilon_V} + \underbrace{\left(\mathcal{R}_p(h^*) - \mathcal{R}^*\right)}_{\text{approximation error } \varepsilon_B}$$

where $h^* = \arg\min_{h \in \mathcal{H}} \mathcal{R}_p(h)$ and $\mathcal{R}^* = \mathcal{R}_p(h_{\text{Bayes}})$.

### Approximation error $\varepsilon_B$ (Bias)

How well the *best* in $\mathcal{H}$ approximates the Bayes predictor.

Determined by the **expressive power** of $\mathcal{H}$.

Independent of the training set.

### Estimation error $\varepsilon_V$ (Variance)

Gap between ERM output and the best in $\mathcal{H}$.

Due to **finite sample**: ERM is only an estimate.

Decreases as $n$ grows.

**Definition 3 (Agnostic PAC with General Loss)**

*$\mathcal{H}$ is agnostic PAC learnable w.r.t. $L : \mathcal{Y} \times \mathcal{Y} \to [0, M]$ if there exist $m_{\mathcal{H}}$ and $\mathcal{A}$ such that for every $\varepsilon, \delta \in (0, 1)$ and $p$, applying $\mathcal{A}$ to $n \geq m_{\mathcal{H}}(\varepsilon, \delta)$ examples returns $h_{\mathcal{T}}$:*

$$\mathcal{R}_p(h_{\mathcal{T}}) = \mathbb{E}_{(\mathbf{x},t)\sim p}[L(h_{\mathcal{T}}(\mathbf{x}), t)] \leq \min_{h \in \mathcal{H}} \mathbb{E}_{(\mathbf{x},t)\sim p}[L(h(\mathbf{x}), t)] + \varepsilon$$

*with probability $\geq 1 - \delta$.*

- Extends the framework to **regression**, cost-sensitive classification, etc.
- Same principles apply — only the loss function changes.
- Bounded loss ($L \leq M$) ensures concentration inequalities hold.

# Representative Samples

## $\varepsilon$-representative training sets

**Key condition for ERM success**

ERM works when empirical risk is a **good proxy** for true risk — i.e. when the training set is *representative*.

**Definition 4 ($\varepsilon$-Representative Sample)**

*$\mathcal{T}$ is $\varepsilon$-representative w.r.t. $\mathcal{H}$, $L$, $p$ if for every $h \in \mathcal{H}$:*

$$\left|\overline{\mathcal{R}}_{\mathcal{T}}(h) - \mathcal{R}_p(h)\right| \leq \varepsilon$$

*Every predictor's training error differs from its true error by at most $\varepsilon$.*

- Rankings are preserved: a better predictor on the population is also better on training data (up to $\varepsilon$).
- If $\mathcal{T}$ is representative, ERM finds a near-optimal predictor.

# Representative Samples
## Why representativeness guarantees PAC

Suppose $\mathcal{T}$ is $\frac{\varepsilon}{2}$-representative. Let $h^* = \arg\min_{h \in \mathcal{H}} \mathcal{R}_p(h)$.

**Step 1**: representativeness applied to $h^*$:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h^*) \leq \mathcal{R}_p(h^*) + \tfrac{\varepsilon}{2}$$

**Step 2**: ERM minimises empirical risk:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) \leq \overline{\mathcal{R}}_{\mathcal{T}}(h^*) \leq \mathcal{R}_p(h^*) + \tfrac{\varepsilon}{2}$$

**Step 3**: representativeness applied to $h_{\mathcal{T}}$:

$$\mathcal{R}_p(h_{\mathcal{T}}) \leq \overline{\mathcal{R}}_{\mathcal{T}}(h_{\mathcal{T}}) + \tfrac{\varepsilon}{2} \leq \mathcal{R}_p(h^*) + \varepsilon$$

**Conclusion**

Representativeness **directly implies** the agnostic PAC property.

**Definition 5 (Uniform Convergence)**

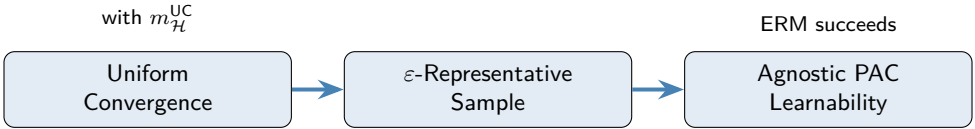
*$\mathcal{H}$ has the **uniform convergence** property w.r.t. $L$ if there exists $m_{\mathcal{H}}^{UC} : (0,1)^2 \to \mathbb{N}$ such that: For every $\varepsilon, \delta \in (0,1)$ and $p$, if $n \geq m_{\mathcal{H}}^{UC}(\varepsilon, \delta)$ pairs are drawn i.i.d. from $p$, then with probability $\geq 1 - \delta$ the sample is $\varepsilon$-representative.*

- Not just one predictor converges — **all** predictors in $\mathcal{H}$ must converge simultaneously.
- Harder than standard statistical convergence.
- For **finite classes**: union bound gives uniform convergence with:

$$m_{\mathcal{H}}^{\mathsf{UC}}(\varepsilon, \delta) \leq \left\lceil \frac{1}{\varepsilon^2} \ln \frac{2|\mathcal{H}|}{\delta} \right\rceil$$

- Uses **Hoeffding / Chernoff** concentration inequalities.

# Uniform Convergence
## The implication chain




### Formal connection

If $\mathcal{H}$ has UC with $m_{\mathcal{H}}^{\mathsf{UC}}$, then $\mathcal{H}$ is agnostic PAC learnable with ERM and sample complexity:

$$m_{\mathcal{H}}(\varepsilon, \delta) \leq m_{\mathcal{H}}^{\mathsf{UC}}\left(\tfrac{\varepsilon}{2}, \delta\right)$$

**Theorem 1**

*Let $\mathcal{H}$ be a finite hypothesis class, $\mathcal{X} \times \mathcal{Y}$ a domain, and $L : \mathcal{X} \times \mathcal{Y} \to [0, 1]$ a bounded loss. Then $\mathcal{H}$ has the uniform convergence property with:*

$$m_{\mathcal{H}}^{UC}(\varepsilon, \delta) \leq \left\lceil \frac{1}{2\varepsilon^2} \ln \frac{2|\mathcal{H}|}{\delta} \right\rceil$$

*Consequently, $\mathcal{H}$ is agnostically PAC learnable via ERM with:*

$$m_{\mathcal{H}}(\varepsilon, \delta) \leq m_{\mathcal{H}}^{UC}\left(\tfrac{\varepsilon}{2}, \delta\right) \leq \left\lceil \frac{1}{\varepsilon^2} \ln \frac{2|\mathcal{H}|}{\delta} \right\rceil$$

- $1/\varepsilon^2$ (agnostic) vs. $1/\varepsilon$ (realizable): agnostic learning needs **more data**.
- Still logarithmic in $|\mathcal{H}|$ — finite classes remain tractable.

**Practical approach for infinite parameterised classes**

Suppose $\mathcal{H}$ is parameterised by $d$ real numbers. In a computer, each parameter uses 64-bit floats: at most $2^{64}$ values per parameter.

Effective size: $|\mathcal{H}_{\text{eff}}| \leq 2^{64d}$.

Applying the finite-class bound:

$$m_{\mathcal{H}}(\varepsilon, \delta) \leq \frac{1}{\varepsilon^2} \ln \frac{2 \cdot 2^{64d}}{\delta} = \frac{128d + 2\ln(2/\delta)}{\varepsilon^2}$$

- Sample complexity grows **linearly** in the number of parameters $d$.
- Rough but practical estimate for many real-world models.
- **Limitation**: depends on machine representation of reals.
- The rigorous treatment uses **VC dimension** (see later).

# Relevance of Inductive Bias

## Does a universal learner exist?

**The question**

Can there exist a **universal learner** $\mathcal{A}^*$ that finds a low-risk predictor for *any* distribution $p$, without any prior assumption?

- Any algorithm implicitly uses prior knowledge by choosing $\mathcal{H}$.
- No inductive bias $\Rightarrow \mathcal{H} = \mathcal{F} =$ all possible functions.
- The question: is $\mathcal{F}$ PAC learnable?

**The answer: No**

The **No-Free-Lunch (NFL) Theorem** shows that for binary classification, no single learning algorithm can succeed on every possible task.

# The No-Free-Lunch Theorem
## Formal statement

**Theorem 2 (No-Free-Lunch)**

*Let $\mathcal{X}$ be a domain, and let $\mathcal{A}$ be any learning algorithm for binary classification with 0-1 loss. For any sample size $n < |\mathcal{X}|/2$, there exists a distribution $\mathcal{D}$ over $\mathcal{X} \times \{0,1\}$ such that:*

1. ***Realizability**: there exists $h^* : \mathcal{X} \to \{0,1\}$ with $\mathcal{R}_{\mathcal{D}}(h^*) = 0$.*
2. ***High error with high probability**: when $\mathcal{A}$ is trained on $S \sim \mathcal{D}^n$,*

$$\mathbb{P}_{S\sim\mathcal{D}^n}\left[\mathcal{R}_{\mathcal{D}}(h_S) \geq \tfrac{1}{8}\right] \geq \tfrac{1}{7}.$$

- For *every* learner $\mathcal{A}$, one can construct an **adversarial** distribution that makes $\mathcal{A}$ fail.
- Even though the task is realizable, error $\geq 1/8$ with probability $\geq 1/7$.
- Constants $1/8$ and $1/7$ are not sharp but the message is qualitative.

# The No-Free-Lunch Theorem

## Implication: all functions not PAC learnable

### Theorem 3

*Let $\mathcal{X}$ be an infinite set, and let $\mathcal{F} = \{f : \mathcal{X} \to \{0, 1\}\}$ be all binary classifiers. Then $\mathcal{F}$ is* ***not*** *PAC learnable.*

### Proof sketch

Assume $\mathcal{F}$ is PAC learnable with algorithm $\mathcal{A}_{\mathcal{F}}$ and complexity $m_{\mathcal{F}}(\varepsilon, \delta)$. Choose $\varepsilon = \frac{1}{16}$, $\delta = \frac{1}{8}$, $n = m_{\mathcal{F}}(\varepsilon, \delta)$.

Since $\mathcal{X}$ is infinite, $n < |\mathcal{X}|/2$. The NFL theorem gives a distribution $\mathcal{D}$ with:

- A perfect $h^*$ (realizability).
- $\Pr[\mathcal{R}_{\mathcal{D}}(\mathcal{A}_{\mathcal{F}}(S)) \geq \frac{1}{8}] \geq \frac{1}{7} > \delta$.

Contradiction with the PAC guarantee ($\varepsilon = \frac{1}{16} < \frac{1}{8}$). $\square$

# The No-Free-Lunch Theorem
## Consequences for learning

**Key conclusion**

Without any inductive bias (using all possible functions), learning is **impossible**. Inductive bias is a **fundamental necessity**, not a convenience.

- Choosing $\mathcal{H} \subsetneq \mathcal{F}$ restricts candidates — avoiding unfavourable distributions.
- But it may **exclude the true predictor**: approximation error may be positive.
- This dilemma is the **Bias–Complexity Tradeoff**.

**Practical implication**

Prior knowledge about the task is not optional.
Successful learning requires a **carefully chosen** $\mathcal{H}$ that balances expressiveness and generalisability.

# Bias–Complexity Tradeoff

## Risk decomposition revisited

### Excess risk decomposition

$$\mathcal{R}_p(h_{\mathcal{T}}) - \mathcal{R}_p(h_{\text{Bayes}}) = \underbrace{\mathcal{R}_p(h_{\mathcal{T}}) - \mathcal{R}_p(h^*)}_{\varepsilon_V \ \text{(variance)}} + \underbrace{\mathcal{R}_p(h^*) - \mathcal{R}_p(h_{\text{Bayes}})}_{\varepsilon_B \ \text{(bias)}}$$

### Bias $\varepsilon_B$ (approximation error)

- Minimum risk in $\mathcal{H}$ vs. Bayes risk.
- Depends on **expressive power** of $\mathcal{H}$.
- Independent of the training set.
- Decreases as $\mathcal{H}$ grows.

### Variance $\varepsilon_V$ (estimation error)

- ERM output vs. best in $\mathcal{H}$.
- Due to **finite sample**.
- Decreases as $n$ grows.
- Increases as $\mathcal{H}$ grows.

# Bias–Complexity Tradeoff

## Effects of model complexity

### High Bias, Low Variance (Underfitting)

$\mathcal{H}$ too simple (e.g. linear for nonlinear data).

**Large** approximation error: best in $\mathcal{H}$ is far from the truth.

**Small** estimation error: similar predictors across training sets.

Both training and test errors are high.

### Low Bias, High Variance (Overfitting)

$\mathcal{H}$ too complex (e.g. deep nets).

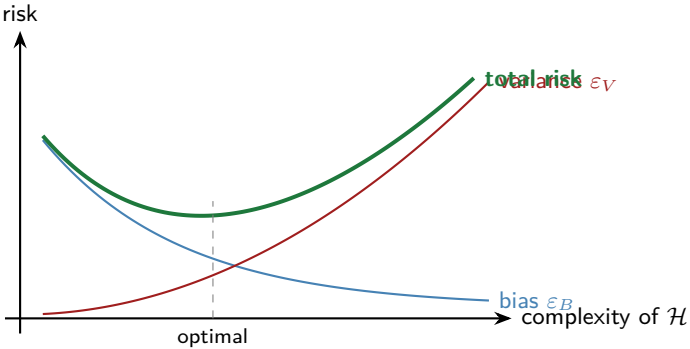
**Small** approximation error: $\mathcal{H}$ may contain the Bayes predictor.

**Large** estimation error: different datasets yield wildly different predictors.

Low training error, high test error.

# Bias–Complexity Tradeoff

## The U-shaped risk curve




*As model complexity increases, bias decreases but variance increases. The total risk has a minimum at the optimal complexity.*

# Bias–Complexity Tradeoff

## Nested hypothesis classes

In practice, consider a **nested family**:

$$\mathcal{H}_1 \subset \mathcal{H}_2 \subset \cdots \subset \mathcal{H}_k$$

Examples:

- Polynomial regression with increasing degree.
- Decision trees with increasing depth.
- Neural networks with increasing width / depth.

As $k$ increases:

- Approximation error (bias) **decreases** monotonically.
- Estimation error (variance) **increases**.
- Total risk typically follows a **U-shaped curve**.
- The optimal $k$ must be chosen via **model selection**.

**Definition**

**Model selection** is the process of identifying the hypothesis class (type of predictor + hyperparameter values) that best balances approximation and estimation error for the given task.

- In practice: a set of predictors parameterised by **hyperparameters**.
- A type of predictor + hyperparameter values $\Rightarrow$ a class of *parametric predictors*.
- ERM (or another algorithm) selects the best predictor *within* the class.
- Model selection evaluates and selects the *class itself*.

- Good design requires classes where *both* bias and variance are not excessively high.
- Domain expertise guides the design of good hypothesis classes.

# VC Dimension

## Why we need a new complexity measure

- Finite classes: learnability characterised by $|\mathcal{H}|$ — straightforward.
- Infinite classes: $|\mathcal{H}| = \infty$ — the counting argument breaks down.

**Example: threshold functions**

$\mathcal{H}_\theta = \{\mathbf{1}[x < \theta] : \theta \in \mathbb{R}\}$ — infinite class.

Yet it is PAC learnable with complexity $m_{\mathcal{H}}(\varepsilon, \delta) \leq \lceil \frac{1}{\varepsilon} \log \frac{2}{\delta} \rceil$.

- Finiteness is **sufficient** but **not necessary** for learnability.
- We need a measure of complexity that captures the *effective* size.
- The answer: **Vapnik–Chervonenkis (VC) Dimension**.

### Restriction of $\mathcal{H}$ to a set $C$

Given $C = \{c_1, \ldots, c_m\} \subset \mathcal{X}$, the **restriction** is:

$$\mathcal{H}_C = \{(h(c_1), \ldots, h(c_m)) : h \in \mathcal{H}\} \subseteq \{0,1\}^{|C|}$$

The set of all labellings of $C$ achievable by predictors in $\mathcal{H}$.

### Shattering

$\mathcal{H}$ **shatters** $C$ if $\mathcal{H}_C$ contains **all** $2^{|C|}$ binary labellings of $C$.

For every possible way to split $C$ into two classes, some $h \in \mathcal{H}$ achieves that split exactly.

# VC Dimension
## Definition

**Definition 6 (VC Dimension)**

*The **VC-dimension** $\mathrm{VCdim}(\mathcal{H})$ is the size of the **largest** subset $C \subset \mathcal{X}$ that is shattered by $\mathcal{H}$.*

*If $\mathcal{H}$ shatters arbitrarily large sets: $\mathrm{VCdim}(\mathcal{H}) = \infty$.*

**To prove $\mathrm{VCdim}(\mathcal{H}) = d$ we need two things:**

1. Show $\mathrm{VCdim}(\mathcal{H}) \geq d$: exhibit a set of $d$ points shattered by $\mathcal{H}$.
2. Show $\mathrm{VCdim}(\mathcal{H}) < d + 1$: prove **no** set of $d + 1$ points is shattered.

# VC Dimension

## Example: threshold functions on $\mathbb{R}$

**Class**: $\mathcal{H}^{\text{thr}} = \{h_\theta : h_\theta(x) = 1 \text{ iff } x \geq \theta,\ \theta \in \mathbb{R}\}$

**Shattering $C = \{c_1\}$ (size 1):**

- Label $1$: set $\theta = c_1 \Rightarrow h_\theta(c_1) = 1$. ✓
- Label $0$: set $\theta = c_1 + 1 \Rightarrow h_\theta(c_1) = 0$. ✓
- $\Rightarrow \mathrm{VCdim}(\mathcal{H}^{\text{thr}}) \geq 1$.

**No shattering of $C = \{c_1, c_2\}$ (size 2), $c_1 \leq c_2$:**

- The labelling $(c_1 = 0, c_2 = 1)$ requires: threshold classifies $c_2$ as $1$ but $c_1$ as $0$.
- Any $\theta$ classifying $c_2 = 1$ (i.e. $\theta \leq c_2$) also classifies $c_1 = 1$. Contradiction.
- $\Rightarrow \mathrm{VCdim}(\mathcal{H}^{\text{thr}}) < 2$.

**Result**

$\mathrm{VCdim}(\mathcal{H}^{\text{thr}}) = 1$.

# VC Dimension

## Example: axis-aligned rectangles in $\mathbb{R}^2$

**Class**: $\mathcal{H}^{\text{rect}}$ = all axis-aligned rectangles; label $1$ if inside.

**Lower bound** ($\mathrm{VCdim}(\mathcal{H}^{\text{rect}}) \geq 4$):
Take 4 points at compass positions on a circle.

- Any subset labelled $1$: enclose only those points with a tight rectangle.
- All $2^4 = 16$ labellings achievable. ✓

**Upper bound** ($\mathrm{VCdim}(\mathcal{H}^{\text{rect}}) < 5$):
Any 5 points: consider their **bounding box**.

- At least one point lies inside the bounding box (or on a side).
- Label the other 4 positive and that one negative.
- Any rectangle containing all 4 positives must also contain the negative. $\times$

**Result**

$\mathrm{VCdim}(\mathcal{H}^{\text{rect}}) = 4.$

# VC Dimension

## Example: intervals on $\mathbb{R}$

**Class**: $\mathcal{H}^{\text{int}} = \{h_{a,b}(x) = \mathbf{1}[a < x < b] : a < b \in \mathbb{R}\}$

**Shattering $C = \{1, 2\}$ (size 2):**

- Label $(1, 0)$: interval $(0.5, 1.5)$. ✓
- Label $(0, 1)$: interval $(1.5, 2.5)$. ✓
- Label $(1, 1)$: interval $(0.5, 2.5)$. ✓
- Label $(0, 0)$: interval $(3, 4)$ (outside both). ✓
- $\Rightarrow \mathrm{VCdim}(\mathcal{H}^{\text{int}}) \geq 2$.

**No shattering of $C = \{c_1, c_2, c_3\}$, $c_1 \leq c_2 \leq c_3$:**

- Labelling $(1, 0, 1)$ impossible: any interval including $c_1$ and $c_3$ must include $c_2$.
- $\Rightarrow \mathrm{VCdim}(\mathcal{H}^{\text{int}}) < 3$.

**Result**

$\mathrm{VCdim}(\mathcal{H}^{\text{int}}) = 2$.

**Upper bound on VC dimension of finite classes**

To shatter a set $C$ we need all $2^{|C|}$ labellings.
Since $|\mathcal{H}_C| \leq |\mathcal{H}|$, shattering requires $|\mathcal{H}| \geq 2^{|C|}$, i.e.:

$$\mathrm{VCdim}(\mathcal{H}^{\mathsf{fin}}) \leq \log_2 |\mathcal{H}|$$

- The VC dimension of a finite class can be **much smaller** than $\log_2 |\mathcal{H}|$.
- Example: $\mathcal{X} = \{1, \ldots, k\}$, threshold class. $|\mathcal{H}| = k$ but $\mathrm{VCdim}(\mathcal{H}) = 1$ — the gap can be arbitrarily large.
- VC dimension is a **tighter** complexity measure than class size.

**Theorem 4 (Fundamental Theorem of Statistical Learning)**

*Let $\mathcal{H}$ be a binary classification class with 0-1 loss. The following are equivalent:*

1. *$\mathcal{H}$ has **finite VC dimension** $d$.*
2. *$\mathcal{H}$ is **agnostic PAC learnable**, with constants $c_1 < c_2$:*

$$\frac{c_1}{\varepsilon^2}\left(d + \ln\frac{1}{\delta}\right) \leq m_{\mathcal{H}}(\varepsilon, \delta) \leq \frac{c_2}{\varepsilon^2}\left(d + \ln\frac{1}{\delta}\right)$$

   *Moreover, ERM is a successful agnostic PAC learner for $\mathcal{H}$.*

3. *$\mathcal{H}$ is **PAC learnable** (under realizability), with:*

$$\frac{c_1}{\varepsilon}\left(d + \ln\frac{1}{\delta}\right) \leq m_{\mathcal{H}}(\varepsilon, \delta) \leq \frac{c_2}{\varepsilon}\left(d + \ln\frac{1}{\delta}\right)$$

   *Moreover, ERM is a successful PAC learner for $\mathcal{H}$.*

**Central message**

For binary classification with 0-1 loss:

**Finite VC dimension $\Leftrightarrow$ Learnability**

- VC dimension $d$ is the *right* measure of complexity for infinite classes.
- Sample complexity scales as $\frac{d}{\varepsilon^2}$ (agnostic) or $\frac{d}{\varepsilon}$ (realizable).
- ERM is optimal (up to constants) whenever learning is possible.
- $d = \infty \Rightarrow$ not PAC learnable (consistent with NFL theorem).

**Agnostic case**

$$m_{\mathcal{H}} = \Theta\left(\frac{d + \ln(1/\delta)}{\varepsilon^2}\right)$$

**Realizable case**

$$m_{\mathcal{H}} = \Theta\left(\frac{d + \ln(1/\delta)}{\varepsilon}\right)$$

# Summary
## Key results

### Finite classes

- Always PAC/agnostic PAC learnable.
- Sample complexity: $\Theta\left(\frac{\ln|\mathcal{H}|}{\varepsilon^{(1 \text{ or } 2)}}\right)$.
- ERM is optimal.
- Union bound + concentration inequalities.

### Infinite classes

- VC dimension $d$ characterises learnability.
- $d < \infty \Leftrightarrow$ PAC learnable.
- Complexity: $\Theta\left(\frac{d}{\varepsilon^{(1 \text{ or } 2)}}\right)$.
- ERM remains optimal.

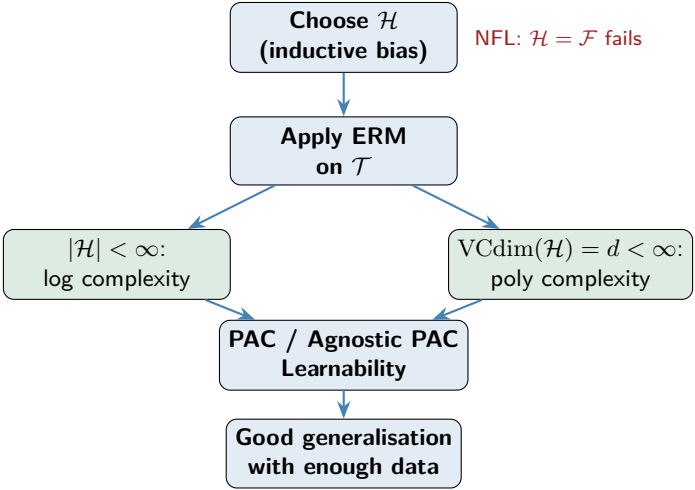
### NFL Theorem

- No universal learner exists.
- All functions not PAC learnable.
- Inductive bias is **necessary**.

### Bias–Complexity

- Excess risk = bias + variance.
- Optimal $\mathcal{H}$ balances both.
- Model selection identifies $\mathcal{H}$.

# Summary
## The big picture



# Summary
## Key formulas at a glance

| Result | Formula / Statement |
|---|---|
| Sample complexity (finite, realizable) | $n \geq \left\lceil \frac{1}{\varepsilon} \ln \frac{\lvert\mathcal{H}\rvert}{\delta} \right\rceil$ |
| UC sample complexity (finite) | $m_{\mathcal{H}}^{\mathsf{UC}} \leq \left\lceil \frac{1}{2\varepsilon^2} \ln \frac{2\lvert\mathcal{H}\rvert}{\delta} \right\rceil$ |
| Agnostic PAC (finite) | $m_{\mathcal{H}} \leq \left\lceil \frac{1}{\varepsilon^2} \ln \frac{2\lvert\mathcal{H}\rvert}{\delta} \right\rceil$ |
| Discretisation (d params) | $m_{\mathcal{H}} \leq \frac{128d+2\ln(2/\delta)}{\varepsilon^2}$ |
| NFL theorem | $\mathcal{F} = \{f : \mathcal{X} \to \{0,1\}\}$ not PAC learnable |
| Risk decomposition | $\mathcal{R}(h_{\mathcal{T}}) - \mathcal{R}^* = \varepsilon_V + \varepsilon_B$ |
| Fundamental Theorem (agnostic) | $m_{\mathcal{H}} = \Theta\left(\frac{d+\ln(1/\delta)}{\varepsilon^2}\right)$ |
| Fundamental Theorem (realizable) | $m_{\mathcal{H}} = \Theta\left(\frac{d+\ln(1/\delta)}{\varepsilon}\right)$ |

# Loss functions

## Course of Machine Learning

Giorgio Gambosi

Master's Degree in Computer Science
University of Rome "Tor Vergata"

A.Y. 2025–2026

# Outline

# The Loss Function — Concept

**Definition**

The **loss function** $\mathcal{L} : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}$ measures the cost of predicting $y$ when the true target is $t$.

- Both $y$ (prediction) and $t$ (target) belong to the output space $\mathcal{Y}$.
- $\mathcal{Y}$ can be: real numbers (regression), class labels (classification), structured outputs.
- For a single input $\mathbf{x}$ with true output $t$, the pointwise quality is:

$$\mathcal{R}(\mathbf{x}, t) = L(h(\mathbf{x}),\, t)$$

- The loss function drives the *definition* of what "good predictions" means.

# Empirical Risk

**Definition**

The **empirical risk** aggregates the loss over the training set $\mathcal{T}$:

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} L(h(\mathbf{x}), t)$$

- Measures the *average* error of $h$ on the available data.
- Lower value $\Rightarrow$ better fit to training data.
- Caveat: minimising empirical risk $\neq$ minimising generalisation error.
- True goal: perform well on *unseen* data (generalisation).

# Parametric Models & Overall Loss

- In practice, $h = h_{\boldsymbol{\theta}}$ where $\boldsymbol{\theta} = (\theta_0, \ldots, \theta_d)$ are learned parameters.
- Training = searching for the $\boldsymbol{\theta}$ minimising:

$$\mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \sum_{i=1}^{n} L_i(\boldsymbol{\theta}) \quad \text{where} \quad L_i = L(\boldsymbol{\theta}; \mathbf{x}_i, t_i)$$

- The overall loss sums individual contributions from each example.

**Interpretation**

Each data point $(\mathbf{x}_i, t_i)$ contributes a term $L_i(\boldsymbol{\theta})$ that penalises the model for predicting incorrectly on that specific example. Training adjusts $\boldsymbol{\theta}$ to reduce this total penalty.

# Goal: Finding a Global Minimum

- We seek a global minimum of $\mathcal{L}(\boldsymbol{\theta}; \mathcal{T})$ over the parameter space.
- A global minimum is a point where the loss is *lower than at every other point*.

**Calculus Approach**

At any minimum (local or global), the gradient must vanish:

$$\nabla \mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \mathbf{0}$$

i.e., each partial derivative satisfies:

$$\frac{\partial}{\partial \theta_i} \mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = 0 \quad \forall\, i$$

- **Gradient** $\nabla f$ at a point: vector of steepest ascent.
- **Negative gradient** $-\nabla f$: direction of steepest descent.

# Challenges of Analytical Solutions

Setting $\nabla \mathcal{L} = \mathbf{0}$ has several critical limitations:

- **Multiple solutions**: critical points include minima, maxima, *and* saddle points.
- **Analytical intractability**: complex models (e.g. neural networks) rarely admit closed-form solutions.
- **Non-linearity**: deep networks compose many non-linear functions — no algebraic formula exists.

**Consequence**

In most practical settings we must resort to **iterative numerical methods**.

# Gradient Descent — Principle

**Core idea**

Move iteratively in the direction of steepest descent of the empirical risk.

- Initialise parameters: $\boldsymbol{\theta}^{(0)} = (\theta_0^{(0)}, \ldots, \theta_d^{(0)})$ (often random).
- At each iteration $i$, update *every* parameter:

$$\theta_j^{(i)} \leftarrow \theta_j^{(i-1)} - \eta \frac{\partial}{\partial \theta_j} \overline{\mathcal{R}}_{\mathcal{T}}(\boldsymbol{\theta}) \Big|_{\boldsymbol{\theta}^{(i-1)}}$$

- $\eta > 0$ is the **learning rate**: controls the step size.

# Gradient Descent — Matrix Form

In compact matrix notation, the update becomes:

$$\boldsymbol{\theta}^{(i)} \leftarrow \boldsymbol{\theta}^{(i-1)} - \frac{\eta}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}), t) \Big|_{\boldsymbol{\theta}^{(i-1)}}$$

- All parameters are updated *simultaneously*.
- The gradient is evaluated on the *entire* training set at each step.
- This variant is called **batch gradient descent**.

**Intuition**

Think of a ball rolling down a landscape: at each step it moves in the direction the slope is steepest, eventually settling in a valley.

# The Learning Rate $\eta$

### Too small $\eta$

- Very slow convergence.
- Reliable but impractical for large models.

### Too large $\eta$

- May overshoot the minimum.
- Can oscillate or diverge.

### Challenges in practice

- Many local minima in non-convex landscapes.
- Saddle points: gradient $= \mathbf{0}$, yet not a minimum.
- Convergence speed depends on curvature and $\eta$.
- Modern solutions: momentum, adaptive rates (Adam, RMSProp).

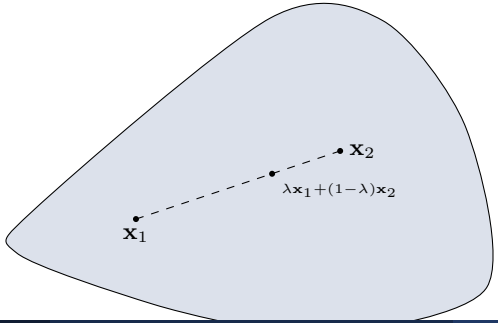
# Convex Sets

## Definition

A set $S \subset \mathbb{R}^d$ is **convex** if for any $\mathbf{x}_1, \mathbf{x}_2 \in S$ and $\lambda \in (0, 1)$:

$$\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in S$$

- Any point on the *segment* joining $\mathbf{x}_1$ and $\mathbf{x}_2$ must lie in $S$.
- Convex sets have no holes or indentations.



# Convex Functions

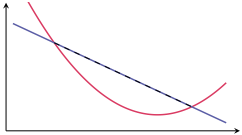
**Definition**

$f : \mathbb{R}^d \to \mathbb{R}$ is **convex** if for all $\mathbf{x}_1, \mathbf{x}_2$ and $\lambda \in (0, 1)$:

$$f(\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2)$$

It is **strictly convex** if the inequality is strict for $\mathbf{x}_1 \neq \mathbf{x}_2$.

- Geometric view: the chord between any two graph points lies *above* the graph.



# Why Convexity Matters

**Key theorem**

If $f$ is **convex**, any local minimum is also a **global minimum**.
If $f$ is **strictly convex**, the global minimum is **unique**.

- For a convex loss, solving $\nabla \mathcal{L}(\boldsymbol{\theta}; \mathcal{T}) = \mathbf{0}$ *guarantees* a global minimum.
- Gradient descent will converge to the optimum regardless of initialisation.
- **Sum rule**: a (positively) weighted sum of convex functions is convex.
- As a consequence, if each $L(\boldsymbol{\theta}; \mathbf{x}_i, t_i)$ is convex, so is $\overline{\mathcal{R}}_{\mathcal{T}}(h_{\boldsymbol{\theta}})$.

**Practical implication**

Designing *convex* loss functions makes optimisation reliable and predictable. Non-convex losses (e.g. in deep networks) require more sophisticated strategies.

# Convexity in Practice

**Favourable cases**

- Linear regression + quadratic loss: *strictly convex*, closed-form solution.
- Logistic regression + log loss: *strictly convex*.
- Linear SVM + hinge loss: *convex*.

**Unfavourable cases**

- Deep neural networks: *highly non-convex*.
- Many saddle points; many local minima.
- Training requires careful initialisation and adaptive optimisers.

- When convexity cannot be guaranteed, we use advanced methods: momentum, second-order approximations, learning-rate schedules, etc.

# Overview: Loss Functions for Regression

- In regression, both $t$ and $h(\mathbf{x})$ are *real-valued*.
- Error = some distance between prediction and target.
- Different distances $\Rightarrow$ different loss functions with different properties.

| **Loss** | **Formula** | **Property** |
|---|---|---|
| Quadratic ($L_2$) | $(y-t)^2$ | Smooth, sensitive to outliers |
| Absolute ($L_1$) | $\lvert y-t \rvert$ | Robust, non-smooth at 0 |
| Huber | piecewise | Best of both worlds |

# Quadratic Loss (Squared Error)

**Definition**

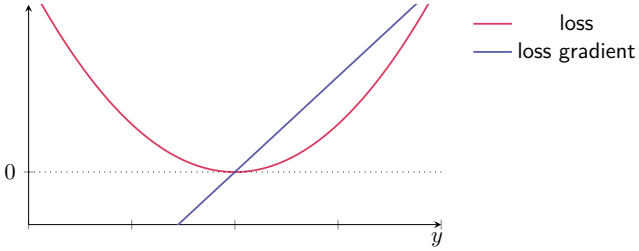
$$L(y, t) = (y - t)^2$$

- Errors penalised *quadratically*: doubling the error quadruples the loss.
- Empirical risk $\Rightarrow$ **Mean Squared Error** (MSE):

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} (h(\mathbf{x}) - t)^2$$

- For **linear regression** $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$:
  - Loss is *strictly convex* in $(\mathbf{w}, b)$.
  - Gradient is *linear* $\Rightarrow$ closed-form solution (normal equations).
- Weakness: very sensitive to outliers.

# Quadratic Loss — Plot




Quadratic loss function and its gradient. Note the parabolic shape and the linear gradient.

# Quadratic Loss — Gradients for Linear Regression

For $h(\mathbf{x}) = \mathbf{w}^T\mathbf{x} + b$, the gradients are:

$$\frac{\partial}{\partial w_i}\mathcal{L} = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t)\, x_i \qquad \frac{\partial}{\partial b}\mathcal{L} = \sum_{(\mathbf{x},t)\in\mathcal{T}} (\mathbf{w}^T\mathbf{x} + b - t)$$

- Gradients are *linear* in the parameters.
- Setting both to zero yields a system of linear equations: the **normal equations**.
- Exact, analytical solution — no iteration required.

**Limitation**

A single large outlier can dominate the sum and distort the entire fitted model.

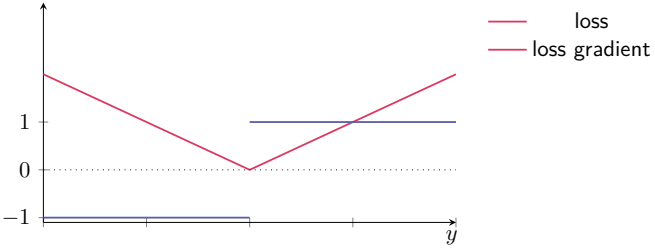
# Absolute Loss ($L_1$)

**Definition**

$$L(y, t) = |y - t|$$

- Errors penalised *linearly*: doubling the error doubles the loss.
- Much more **robust to outliers** than quadratic loss.
- Gradient: piecewise constant, $\pm 1$ — steps of uniform size during GD.
- Convex but *not* strictly convex; gradient undefined at $y = t$.
- Multiple optimal solutions possible; handled via **subgradient methods**.

# Absolute Loss — Plot




Absolute loss function and its gradient. The V-shape is characteristic, with constant gradient magnitude.

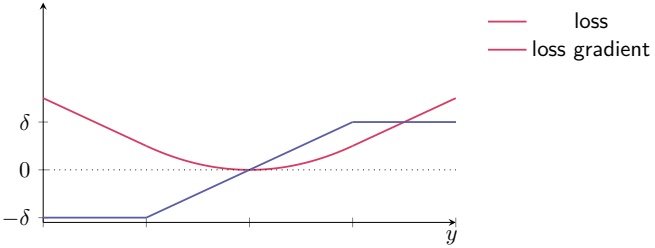
# Huber Loss

**Definition**

$$L(t,y) = \begin{cases} \frac{1}{2}(t-y)^2 & |t-y| \leq \delta \\ \delta\left(|t-y| - \frac{\delta}{2}\right) & |t-y| > \delta \end{cases}$$

- **Quadratic** for small errors $\Rightarrow$ smooth gradients, efficient optimisation.
- **Linear** for large errors $\Rightarrow$ robust to outliers.
- $\delta$ is a hyperparameter controlling the transition point.
- Combines the best properties of $L_2$ and $L_1$.

# Huber Loss — Plot




Huber loss combines quadratic behaviour near zero with linear growth for large errors.

# Comparison: Regression Losses

| | $L_2$ | $L_1$ | **Huber** |
|---|---|---|---|
| Formula | $(y-t)^2$ | $\lvert y-t \rvert$ | piecewise |
| Convexity | strictly | convex | strictly |
| Smoothness | everywhere | not at $y=t$ | everywhere |
| Outlier robustness | low | high | high |
| Gradient descent | direct | subgradient | direct |
| Closed-form (linear) | ✓ | × | × |

# Classification — Setup

- Goal: assign inputs to *discrete* classes.
- Focus on **binary classification**: $t \in \{-1, +1\}$.
- Classifier outputs a real value $y = h(\mathbf{x})$:
  - sign$(y)$ determines the predicted class.
  - $|y|$ encodes prediction confidence.
- Two approaches:

**Direct prediction**

Returns a class label $\hat{t}$.
Error if $\hat{t} \neq t$.

**Probabilistic prediction**

Returns $p(t|\mathbf{x})$ for each class.
Error measured via distribution divergence.

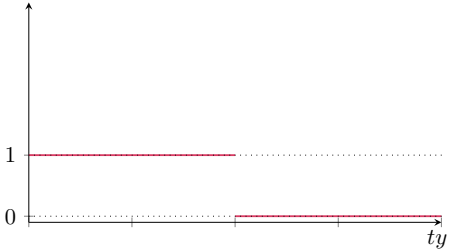
# 0/1 Loss

**Definition**

$$L(t, y) = \mathbb{1}[ty < 0] \quad \Leftrightarrow \quad L(t, y) = \begin{cases} 1 & \text{sgn}(y) \neq t \\ 0 & \text{sgn}(y) = t \end{cases}$$

- Directly minimises classification error rate. Intuitive.
- Critical problems:
  - **Non-convex**: many local minima; no optimality guarantee.
  - **Non-smooth**: discontinuous jump at $ty = 0$.
  - **Zero gradient almost everywhere**: gradient descent cannot progress.
  - **NP-hard**: minimising $\overline{\mathcal{R}}$ for linear classifiers is computationally intractable.

# 0/1 Loss — Plot




The 0/1 loss is a step function that assigns uniform cost to all errors.

# Convex Surrogate Loss Functions

## Motivation

0/1 loss is ideal but intractable. We replace it with a **surrogate** that is:

1. **Upper bound** on 0/1 loss: $L_{\text{surr}}(t, y) \geq \mathbb{K}[ty < 0]$
2. **Convex** in $y$ (and in parameters).
3. **Smooth**: well-defined derivatives almost everywhere.

## Key surrogates

Perceptron loss, Hinge loss, Log loss (cross-entropy), Squared loss (classification), Exponential loss.

Minimising a surrogate $\Rightarrow$ tends to minimise classification error indirectly.

# Perceptron Loss

**Definition**

$$L(t, y) = \max(0, -ty)$$

- **Correctly classified** ($ty > 0$): loss $= 0$. No penalty for correct predictions.
- **Misclassified** ($ty < 0$): loss $= |ty|$. Severity grows with confidence of error.
- Continuous; gradient defined almost everywhere.
- Convex but *not* strictly convex.
- Not a proper surrogate: does not always upper-bound 0/1 loss.
- Historical significance: underlies the **classical Perceptron algorithm** (1950s).

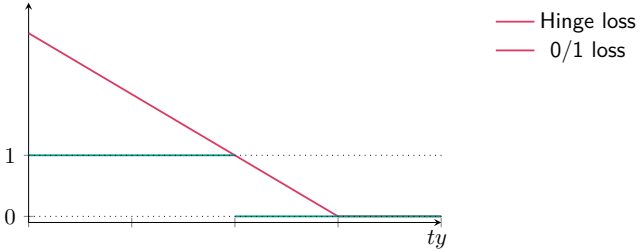
# Hinge Loss

**Definition**

$$L(t, y) = \max(0,\, 1 - ty)$$

- Loss $= 0$ only when $ty \geq 1$ (correct *and* confident).
- $0 < ty < 1$: correct but not confident — still penalised.
- $ty < 0$: misclassified — loss grows linearly.
- Properties: continuous, convex, proper surrogate (upper-bounds 0/1 loss).
- Not differentiable at $ty = 1$ (corner); handled via **subgradients**.
- Foundation of **Support Vector Machines (SVM)**.

# Hinge Loss — Plot



# Non-Differentiable Losses — The Problem

Hinge loss: $L_H(t, y) = \max(0, 1 - ty)$

$$\frac{\partial}{\partial y} L_H = \begin{cases} -t & ty < 1 \\ 0 & ty > 1 \\ \text{undefined} & ty = 1 \end{cases}$$

- The gradient is undefined at the corner $ty = 1$.
- Standard gradient descent cannot be applied directly.
- Same issue arises for absolute loss at $y = t$ and perceptron loss at $ty = 0$.

**Solution: subgradients**

Generalise the gradient to non-smooth points using a *set* of valid linear bounds.

# Subgradients — Definition

For a convex function $f$, at a differentiable point $x$:

$$f(x') \geq f(x) + \nabla f|_x \cdot (x' - x) \quad \forall\, x'$$

(Gradient defines a tangent lower bound.)

**Subgradient**

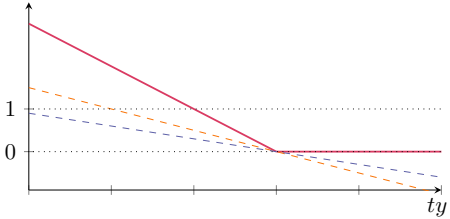
A vector $\bar{\nabla} f|_x$ is a **subgradient** at $x$ if:

$$f(x') \geq f(x) + \bar{\nabla} f|_x \cdot (x' - x) \quad \forall\, x'$$

- At differentiable points: the subgradient is unique and equals the gradient.
- At non-differentiable points: there is a *set* of valid subgradients.
- Any element of this set can be used in place of the gradient in GD.

# Subgradient of Hinge Loss

At the corner $ty = 1$, valid slopes lie between $-t$ and $0$.




A common choice: set the subgradient to $0$ at $ty = 1$, giving:

$$\bar{\nabla}_y L_H = \begin{cases} -t & ty < 1 \\ 0 & ty \geq 1 \end{cases}$$

# Squared Loss for Classification

**Definition**

$$L(t, y) = (1 - ty)^2$$

**Advantages**

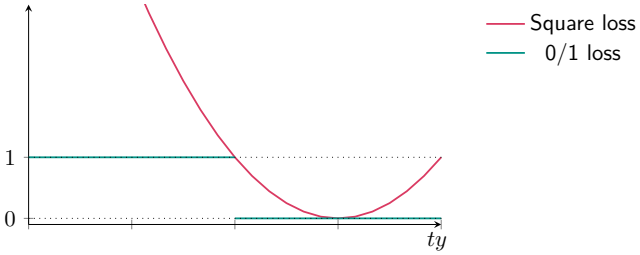
- Infinitely differentiable.
- Convex, well-defined gradient.
- No corners or undefined points.

**Disadvantages**

- *Not* a proper surrogate: does not upper-bound 0/1 loss.
- Sensitive to outliers (quadratic growth).
- Penalises *very confident correct* predictions.

- Rarely the preferred choice in classification practice.

# Squared Loss for Classification — Plot



# Log Loss (Cross-Entropy Loss)

**Definition**

$$L(t, y) = \frac{1}{\log 2} \log\left(1 + e^{-ty}\right)$$

- Also called **logistic loss** or **cross-entropy loss**.
- Standard loss for probabilistic classification and logistic regression.
- Smooth approximation to hinge loss (no corner).
- For large negative $ty$: grows approximately linearly.
- For large positive $ty$: approaches 0 asymptotically (never reaches it).

# Log Loss — Properties

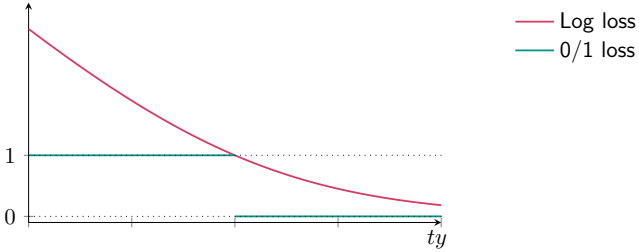
**Mathematical properties**

- Infinitely differentiable everywhere.
- **Strictly convex**: unique global minimum.
- Proper surrogate: upper-bounds 0/1 loss.
- Gradient continuous and well-defined.

**Caution**

- Grows unboundedly for $ty \to -\infty$.
- Sensitive to severely mislabelled examples.
- Growth is *logarithmic*, so more robust than squared loss.

# Log Loss — Plot



# Information Theory Background

**Entropy $H(p)$**

$$H(p) = -\mathbb{E}_{x\sim p}[\log p(x)]$$

Uncertainty / information content of $p$.
Min. avg. bits to encode from $p$.

**Cross-entropy $\mathrm{H}(p,q)$**

$$\mathrm{H}(p,q) = -\mathbb{E}_{x\sim p}[\log q(x)]$$

Avg. bits using code for $q$, when true dist. is $p$.

**KL divergence**

$$\mathrm{KL}(p\|q) = \mathrm{H}(p,q) - H(p)$$

Extra bits wasted by using $q$ instead of $p$.
Always $\geq 0$; $= 0$ iff $p = q$.

- $\mathrm{KL}(p\|q) = 0 \Leftrightarrow p = q$ (almost everywhere).
- Minimising cross-entropy $\equiv$ minimising KL divergence (since $H(p)$ is fixed).

# Cross-Entropy for Binary Classification

Let $p(\mathbf{x}) \in \{0, 1\}$ be the true label, $y(\mathbf{x}) \in [0, 1]$ the predicted probability for class $C_1$.

**Empirical cross-entropy**

$$\mathrm{CE}(\mathcal{T}) = -\frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \Big[ t \log y(\mathbf{x}) + (1-t)\log(1-y(\mathbf{x})) \Big]$$

- For each example: $\log$ probability assigned to the *correct* class, averaged.
- The negative sign converts high probability (good) into low loss.

Separating by class:

$$\mathrm{CE}(\mathcal{T}) = -\frac{1}{|\mathcal{T}|} \left( \sum_{(\mathbf{x},t)\in C_1} \log y(\mathbf{x}) + \sum_{(\mathbf{x},t)\in C_0} \log(1-y(\mathbf{x})) \right)$$

# Log Loss $\equiv$ Cross-Entropy (Logistic Regression)

For logistic regression: $y(\mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \dfrac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}}$

Substituting into cross-entropy, using $t \in \{-1, +1\}$:

$$\mathrm{CE}(\mathcal{T}) = \frac{1}{|\mathcal{T}|} \sum_{(\mathbf{x},t)\in\mathcal{T}} \log\Big(1 + e^{-t(\mathbf{w}^T\mathbf{x}+b)}\Big)$$

**Connection to log loss**

$$\overline{\mathcal{R}}_{\mathcal{T}}(h) = \frac{1}{|\mathcal{T}| \log 2} \sum_{(\mathbf{x},t)\in\mathcal{T}} \log\Big(1 + e^{-t(\mathbf{w}^T\mathbf{x}+b)}\Big)$$

- Minimising log loss $\equiv$ minimising cross-entropy.
- Strong information-theoretic justification for using log loss in classification.

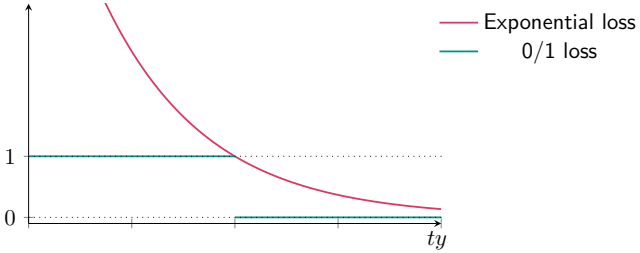
# Exponential Loss

**Definition**

$$L(t, y) = e^{-ty}$$

- Prominent in **boosting** algorithms, especially **AdaBoost**.
- Penalises wrong predictions more aggressively than log loss: exponential growth.
- Infinitely differentiable everywhere.
- Strictly convex $\Rightarrow$ unique global minimum.
- Proper surrogate: upper-bounds $0/1$ loss.
- Sensitive to outliers: very large penalty for confident misclassifications.

# Exponential Loss — Plot



# Comparison: Classification Losses

| | **0/1** | **Perceptron** | **Hinge** | **Log** | **Exp.** |
|---|---|---|---|---|---|
| Upper-bounds 0/1 | — | partial | ✓ | ✓ | ✓ |
| Convex | No | Yes | Yes | Yes | Yes |
| Strictly convex | No | No | No | ✓ | ✓ |
| Smooth | No | No | No | ✓ | ✓ |
| Outlier robust | — | medium | medium | medium | low |
| Use case | ideal | Perceptron | SVM | Logistic Reg. | AdaBoost |

# Stochastic Gradient Descent (SGD)

## Motivation

Batch GD computes the gradient over the *entire* training set at each step. This is expensive when $|\mathcal{T}|$ is large.

## SGD idea

Use a single randomly chosen example $(\mathbf{x}_i, t_i)$ to estimate the gradient:

$$\boldsymbol{\theta}^{(i)} \leftarrow \boldsymbol{\theta}^{(i-1)} - \eta \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}_i), t_i) \Big|_{\boldsymbol{\theta}^{(i-1)}}$$

- Noisy gradient estimate, but very fast per step.
- Noise can *help* escape local minima.
- Widely used in deep learning practice.

# Mini-Batch Gradient Descent

## Compromise

Use a **mini-batch** $\mathcal{B} \subset \mathcal{T}$ of $m$ randomly chosen examples:

$$\boldsymbol{\theta}^{(i)} \leftarrow \boldsymbol{\theta}^{(i-1)} - \frac{\eta}{m} \sum_{(\mathbf{x},t)\in\mathcal{B}} \nabla L(h_{\boldsymbol{\theta}}(\mathbf{x}), t) \Big|_{\boldsymbol{\theta}^{(i-1)}}$$

| | **Batch** | **Mini-batch** | **SGD** |
|---|---|---|---|
| Gradient quality | exact | good estimate | noisy |
| Speed per step | slow | moderate | fast |
| Memory use | full | moderate | minimal |
| Typical use | small $n$ | deep learning | large $n$ |

# SGD — Key Properties

**Advantages**

- Each update is $O(d)$ — independent of $n$.
- Can process data in a streaming fashion.
- Noise helps escape poor local minima.
- Enables learning from very large datasets.

**Challenges**

- High variance in gradient estimates.
- May not converge to exact minimum.
- Learning rate tuning more critical.
- Convergence guarantees are weaker.

**Modern practice**

Almost all deep learning uses mini-batch SGD with adaptive learning rates (Adam, RMSProp, AdaGrad).

# Summary — Loss Functions

## Core definitions

- **Loss** $L(y, t)$: pointwise prediction penalty.
- **Empirical risk** $\overline{\mathcal{R}}_{\mathcal{T}}(h)$: average loss over training set.
- **Training** $\equiv$ minimising $\overline{\mathcal{R}}_{\mathcal{T}}(h_{\boldsymbol{\theta}})$ over parameters $\boldsymbol{\theta}$.

## Regression losses

- Quadratic ($L_2$): smooth, convex, sensitive to outliers.
- Absolute ($L_1$): robust, requires subgradients.
- Huber: best of both; controlled by $\delta$.

## Classification losses

- 0/1: intuitive but NP-hard to optimise.
- Surrogates (Hinge, Log, Exponential): convex, tractable.
- Log loss $\equiv$ cross-entropy: information-theoretic grounding.

# Central Message

**The loss function encodes what "good predictions" means.**

- Choosing the right loss is as important as choosing the model.
- **Regression**: prefer $L_2$ for smooth data, $L_1$/Huber for outliers.
- **Classification**: use convex surrogates; log loss has the strongest theoretical backing.
- **Optimisation**: exploit convexity when possible; use SGD variants in practice.
- The interplay of loss, optimiser, and model defines the entire learning procedure.